

Problema e escopo do projeto

O projeto **Academia do Edução e do Luizão** foi desenvolvido com o objetivo de solucionar um problema comum em academias de pequeno porte: o controle de alunos, instrutores e treinos realizado por meio de planilhas ou anotações.

Esse tipo de controle pode dificultar a organização das informações, a consulta aos dados dos alunos, o acompanhamento dos treinos e a atualização dos exercícios.

O sistema busca centralizar essas informações em uma aplicação Java, permitindo trabalhar com conceitos de Programação Orientada a Objetos, como **classes, objetos, construtores, métodos, encapsulamento e validação de dados**.

O escopo inicial inclui cadastro de alunos, instrutores e treinos, associação de treinos aos alunos, consulta de treinos, atualização de informações, pesquisa de alunos e exclusão de registros. Essas funcionalidades estão descritas no README do projeto.

1. Classes principais

As principais classes identificadas no desenvolvimento são:

Aluno

Representa uma pessoa matriculada na academia.

A classe possui informações como:

- Nome;
- Idade;
- Telefone;
- CPF, na versão com validação.

Também possui comportamentos relacionados ao aluno, como visualizar o treino e atualizar seus dados.

Na implementação inicial, os atributos são privados e o acesso acontece por meio de métodos da própria classe.

Instrutor

Representa o profissional responsável pelos alunos e pela criação dos treinos.

Possui informações como:

- ID;
- Nome;
- CREF;
- Especialidade.

Entre seus comportamentos estão cadastrar e atualizar treinos.

Treino

Representa o programa de exercícios realizado pelo aluno.

Possui informações como:

- ID;
- Nome;
- Descrição;
- Duração.

Também possui métodos para adicionar e remover exercícios e para exibir os dados do treino.

Academia

Representa a academia e concentra operações como cadastro de alunos, cadastro de instrutores e pesquisa de alunos.

Essa classe funciona como uma representação do ambiente onde os demais objetos são utilizados.

Frequencia

Na etapa de desenvolvimento incremental, foi criada a classe **Frequencia**, responsável pelo controle de presenças.

Ela recebe a quantidade de presenças e o total de aulas e calcula a porcentagem de frequência do aluno.

Por exemplo, no código é utilizado o cenário de **18 presenças em 20 aulas**, resultando em uma frequência de 90%.

Evolucao

Também foi criada a classe **Evolucao**, responsável por representar dados relacionados ao acompanhamento físico, como peso e altura do aluno.

2. Demonstração do sistema funcionando

A demonstração pode ser feita utilizando a versão de **construtores e métodos**.

Primeiramente, o programa cria um objeto **Academia**, um objeto **Aluno**, um **Instrutor** e um **Treino**.

Em seguida, o sistema executa algumas operações:

1. Cadastra o aluno;
2. Cadastra o instrutor;
3. O instrutor cadastra um treino;
4. Adiciona exercícios ao treino;
5. O aluno visualiza seu treino;
6. O treino é exibido na tela;
7. O sistema realiza uma pesquisa pelo nome do aluno.

O próprio **Main.java** apresenta esse fluxo utilizando exemplos como o aluno **Luiz**, o instrutor **Eduardo** e o treino **Treino A**, com exercícios como supino reto, desenvolvimento e tríceps pulley.

Uma segunda demonstração pode ser feita utilizando o desenvolvimento incremental. Nessa versão, o sistema apresenta:

- Dados do aluno;
- Treino;
- Frequência;
- Evolução física.

O programa utiliza, por exemplo, 18 presenças em 20 aulas e dados de peso e altura para demonstrar essas funcionalidades.

3. Explicação do encapsulamento aplicado

O **encapsulamento** foi aplicado principalmente através da utilização do modificador **private** nos atributos das classes.

Isso significa que os dados internos dos objetos não ficam disponíveis para alteração direta por qualquer parte do programa.

Um exemplo mais completo está na classe `Aluno`, da etapa de **encapsulamento e validação**.

Os atributos são definidos como:

```
private String nome;  
private int idade;  
private String cpf;
```

O acesso aos dados ocorre através de métodos `get` e `set`.

Além de controlar o acesso, os métodos `set` também realizam **validações**.

Por exemplo:

- O nome não pode ser vazio;
- A idade deve ser maior que zero;
- O CPF deve possuir exatamente 11 dígitos.

Caso um valor inválido seja informado, o sistema lança uma `IllegalArgumentException`.

Isso é importante porque evita que o objeto `Aluno` fique armazenando dados inválidos.

Um ponto positivo da implementação é que o construtor também utiliza os métodos `setNome`, `setIdade` e `setCpf`. Dessa maneira, a validação é aplicada inclusive no momento da criação do objeto.

4. Decisões tomadas

Durante o desenvolvimento, algumas decisões importantes foram tomadas.

Utilização de Java

Foi escolhida a linguagem Java porque ela permite aplicar de forma clara os conceitos de Programação Orientada a Objetos exigidos no projeto.

Desenvolvimento incremental

O projeto foi construído em etapas.

No diretório de código existem versões relacionadas a:

- Modelo inicial;
- Construtores e métodos;

- Encapsulamento e validação;
- Desenvolvimento incremental.

Isso mostra uma evolução gradual do sistema, começando com uma estrutura mais simples e adicionando novos conceitos e funcionalidades ao longo do desenvolvimento.

Separação das responsabilidades

Cada classe representa uma entidade ou responsabilidade diferente.

Por exemplo:

- `Aluno` representa o aluno;
- `Instrutor` representa o profissional;
- `Treino` representa o treino;
- `Frequencia` controla a frequência;
- `Evolucao` representa o acompanhamento físico.

Essa divisão facilita o entendimento do código e permite que novas funcionalidades sejam adicionadas posteriormente.

Validação dos dados

Foi decidido validar informações importantes antes de armazená-las.

Isso pode ser observado principalmente na classe `Aluno`, onde nome, idade e CPF possuem regras de validação.

Testes

O projeto também possui arquivos `Testes.java` em diferentes etapas e um arquivo `executar-testes.bat`, demonstrando a preocupação em verificar o comportamento das implementações.

5. Dificuldades encontradas

Uma das principais dificuldades foi transformar o problema real de uma academia em classes e responsabilidades dentro da programação orientada a objetos.

Foi necessário identificar quais informações pertenciam a cada entidade e quais comportamentos deveriam ficar dentro de cada classe.

Outra dificuldade foi evoluir o projeto sem perder a organização. O repositório apresenta diferentes etapas de desenvolvimento, começando pelo modelo inicial e posteriormente adicionando construtores, métodos, encapsulamento, validações, frequência e evolução.

Também foi necessário lidar com a **validação dos dados**. Por exemplo, não basta simplesmente armazenar a idade ou o CPF: o sistema precisa impedir valores inválidos. Essa preocupação aparece claramente na implementação da classe **Aluno**.

Outra dificuldade foi representar funcionalidades que existem no escopo do projeto, mas que ainda estão em diferentes níveis de implementação. O README apresenta funcionalidades como associação de treinos aos alunos, pesquisa, atualização e exclusão, enquanto as versões atuais do código demonstram principalmente o modelo das entidades, seus métodos, validações e o desenvolvimento incremental.

Conclusão

O projeto demonstra a evolução de uma aplicação Java para gerenciamento de uma academia, começando por um modelo simples e avançando para conceitos importantes de Programação Orientada a Objetos.

O principal destaque é a evolução para o **encapsulamento com validação**, evitando que informações inválidas sejam armazenadas nos objetos.

Além disso, a divisão em classes como **Aluno**, **Instrutor**, **Treino**, **Academia**, **Frequencia** e **Evolucao** permite representar diferentes partes do problema de maneira organizada.

Como próximos passos, o sistema poderia evoluir para uma aplicação mais completa, implementando de forma integrada o cadastro real dos registros, associação entre alunos e treinos, persistência dos dados em banco de dados, autenticação de usuários e uma interface gráfica ou web.